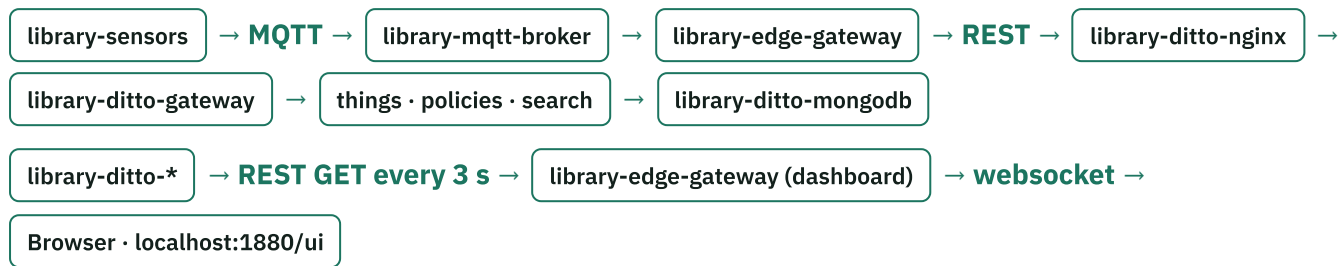


From Sensor to Dashboard

Every step a reading goes through, and what each container does along the way.



The walkthrough follows one real reading from **R02 Computer Lab** and names the container doing the work at each step.

Phase 0 • Start-up (happens once, before any data)

Everything is started with `docker compose up -d`.

Step	Container	What happens
0.1	Docker	Creates a private network <code>smart-library_default</code> . Containers reach each other by service name (<code>mosquitto</code> , <code>nginx</code> , <code>gateway</code> ...). Only ports 1880, 1883 and 8080 are opened to the laptop.
0.2	library-ditto-mongodb	MongoDB starts: the database where Ditto stores everything. Its data lives in the volume <code>smart-library_mongo-data</code> , so twins survive restarts.
0.3	library-ditto-policies library-ditto-things library-ditto-search library-ditto-gateway	The four Ditto Java services start, find each other through the shared network name <code>ditto-cluster</code> and form one cluster. Each reports health on <code>/alive</code> , and Docker marks it healthy .
0.4	library-ditto-nginx	Starts only after the Ditto gateway is healthy. From then on it is the only door into Ditto (port 8080), and it asks for a login.
0.5	library-twin-setup	Runs <code>create_twins.py</code> . Waits until Ditto answers HTTP 200, then sends a <code>PUT</code> for <code>library:R01</code> , <code>library:R02</code> , <code>library:R03</code> (attributes: name, floor, capacity, sensor serial; features empty) and <code>library:gateway</code> . Ditto also creates a policy per twin allowing user <code>ditto</code> to read and write it. The job exits with code 0.
0.6	library-mqtt-broker	Mosquitto starts only after twin-setup succeeded, and listens on port 1883.
0.7	library-edge-gateway	Node-RED starts with the flow baked into its image, connects to <code>mosquitto:1883</code> , subscribes to <code>library/+/telemetry</code> and starts the dashboard on port 1880.
0.8	library-sensors	The simulator starts and connects to the broker as client <code>library-simulator</code> .

This order guarantees the twins exist before the first reading arrives. Compose enforces it with `depends_on` and health checks.

Phase 1 • The sensor creates a reading

`library-sensors` `sensors/simulator.py`

1 Generate values. Once every second, for each room, `reading(room, step)` builds a reading from the room's 240-step script. The script decides whether R02 is normal, busy, or has a broken computer right now. A little random noise is added to make the values realistic.

2 Use the vendor's format. R02 is "vendor B": temperature in Fahrenheit, equipment as codes (0 = OFF, 1 = ON, 2 = FAULT).

```
{"room":"R02","sensor":"LIB-SN-0002","occupancy":22,"humidity":49.0,
  "temperature_f":73.4,"equipment":
  {"lights":1,"ac":1,"computers":2,"printer":1},"ts":1791108000000}
```

R01 and R03 instead send `"temperature": 23.4` and `"ON" / "FAULT"` as text.

3 Sometimes break it on purpose. With an 8 % chance, `break_it()` damages the reading. It can:

- cut the JSON in half
- set occupancy to -5
- set temperature to 70 °C (158 °F for R02)
- set humidity to 140
- remove a field
- put `"abc"` as humidity

4 Publish over MQTT. The JSON string is sent to topic `library/R02/telemetry` at QoS 0 (send once, no acknowledgement).

Phase 2 • The broker delivers it

`library-mqtt-broker` Mosquitto

5 Receive. Mosquitto receives the message over TCP on port 1883. It does not look inside the message or change it.

6 Match subscribers. It checks who subscribed to a matching topic. The gateway subscribed to `library/+/telemetry`, where `+` matches any room, so it matches.

7 Forward. It forwards the message to the gateway immediately. Nothing is stored; if no one were subscribed, the message would simply be lost.

Phase 3 • The edge gateway processes it

`library-edge-gateway` Node-RED, `gateway/flows.json`. Each step below is one node in the flow.

8 MQTT-in node `library/+/telemetry`. Receives the message as a text string: `msg.topic = library/R02/telemetry`, `msg.payload` = the raw string.

9

Function node "Validate & convert" (cleaning).

1. Adds 1 to the **raw** counter.
2. Parses the JSON. If it is broken, the reading is dropped and the **dropped** counter goes up.
3. Takes the room name from the topic (R02).
4. **Converts** Fahrenheit to Celsius: $(73.4 - 32) \times 5/9 = 23.0 \text{ }^{\circ}\text{C}$.
5. **Checks** occupancy, temperature and humidity are real numbers in possible ranges: occupancy 0–500, temperature –10 to 50 °C, humidity 0–100 %. Anything else is dropped and counted; the node shows e.g. "dropped: impossible occupancy –5".
6. **Converts equipment codes** 0 → OFF, 1 → ON, 2 → FAULT and lists the faulty devices.
7. Adds 1 to the **valid** counter and outputs the **common format**:

```
{"room":"R02","occupancy":22,"temperature":23.0,"humidity":49,"equipment":"FAULT",
  "devices":{"lights":"ON","ac":"ON","computers":"FAULT","printer":"ON"},"faults":
  ["computers"]}
```

10

Function node "Window features & status" (collecting). The reading is added to R02's buffer in the node's memory. The node status shows the fill level, e.g. R01 4/10 R02 7/10 R03 9/10 . **Until the buffer holds 10 readings, nothing more happens:** the message stops here.

11

Same node: the 10th reading arrives, features are calculated. The buffer is then emptied for the next window.

Feature	Example (R02)
Average and peak occupancy	21.7 and 24
% of capacity	21.7 / 40 = 54.3 %
Average temperature and humidity	23.2 °C, 49 %
Abnormal count (readings in the window breaking a rule)	10 / 10
Equipment health (devices in FAULT in the latest reading)	1 (computers)

12

Same node: classify. Rules are checked in this order:

- **CRITICAL** if occupancy is over 100 % of capacity, **or** any device is in FAULT, **or** temperature **and** humidity are both outside their limits.
- Otherwise **WARNING** if temperature is outside 20–26 °C, **or** humidity is outside 40–60 %, **or** occupancy is 80–100 %.
- Otherwise **NORMAL**.

R02 has a computer in FAULT, so it is **CRITICAL** with the reason "equipment FAULT: computers".

- 13 Same node: trend.** The window average is added to the room's last 4 averages.
- If each average is higher than the one before **and** the room is above 70 % → seats are **LIMITED**.
 - At 100 % or more → **FULL**.
 - Otherwise → **AVAILABLE**.

The node adds 1 to the **windows** counter and outputs one **result** message. That result goes down **two wires at the same time**: to the decision (step 14) and to the twin update (step 18).

Phase 4 • The edge decision: summary or alert

`library-edge-gateway`

- 14 Switch node "CRITICAL?".** Looks at the result's status. CRITICAL goes to the alert path; everything else goes to the summary path.
- 15a Summary path (NORMAL or WARNING).** Builds a small summary (room, status, reasons, occupancy, %, temperature, humidity, seats), publishes it to `library/R02/summary` through the broker, and adds 1 to the **summaries** counter.
- 15b Alert path (CRITICAL).** Adds 1 to the **alerts** counter and sends three outputs at once:
1. **MQTT alert** to `library/alerts`, so any other system could listen. A debug node also prints it into `docker logs library-edge-gateway`.
 2. **Popup**: "CRITICAL ALERT – R02 Computer Lab: equipment FAULT: computers" appears on every open dashboard.
 3. **Alert list**: added to "Latest CRITICAL alerts", which keeps the last 8.
- In **v2** of the gateway image, the alert also carries an action, e.g. "Call technician for: computers".
- 16 Back through the broker.** Mosquitto delivers the summary or alert to anyone subscribed. In the demo that is `compare_traffic.py` when it runs.
- 17 Data reduction.** 10 raw messages came in; only 1 message goes out. That is the "about 91 % less data" shown on the dashboard.

Phase 5 • Updating the digital twin

18

library-edge-gateway

Node "Build merge-patch". Turns the result into a Ditto request:

- **PATCH** `http://nginx/api/2/things/library:R02`
- Header `Content-Type: application/merge-patch+json`
- Header `Authorization: Basic` (ditto:ditto, base64-encoded)

```
{"features":{
  "occupancy":{"properties":
    {"average":21.7,"peak":24,"percent":54.3,"capacity":40,"trend":"STABLE"}},
  "environment":{"properties":{"temperature":23.2,"humidity":49}},
  "equipment":{"properties":
    {"lights":"ON","ac":"ON","computers":"FAULT","printer":"ON","faults":1}},
    "status":{"properties":{"value":"CRITICAL","reasons":["equipment FAULT:
computers"],"abnormalCount":10}},
    "SeatAvailability":{"properties":{"value":"AVAILABLE"}}}}
```

"Merge-patch" means: change only the fields sent and keep everything else (such as the attributes).

19

library-edge-gateway

Node "PATCH twin". The HTTP request goes over the Docker network to the `nginx` container.

20

library-ditto-nginx

Login check. Compares username and password with its `nginx.htpasswd` file.

- Wrong → answers **401**; the request never reaches Ditto (a pytest test checks this).
- Correct → adds the header `x-ditto-pre-authenticated: nginx:ditto` ("this is user ditto") and forwards the request to `gateway:8080`.

21

library-ditto-gateway

Ditto's REST front end. Reads the user (`nginx:ditto`), checks the JSON, turns the HTTP request into an internal "merge this Thing" command and sends it through the cluster to the Things service.

22

library-ditto-things

library-ditto-policies

Permission check. The Things service looks up the twin's policy, which the Policies service manages, and asks: may `nginx:ditto` **write** to `library:R02`? Yes, because twin-setup created that policy in step 0.5.

23

library-ditto-things

library-ditto-mongodb

Apply the change. Merges the new feature values into the twin, increases the twin's revision number, saves the change in MongoDB and answers **204 No Content** (success). The answer travels back: Ditto gateway → nginx → Node-RED.

24

library-ditto-search

Update the search index. Is told about the change and updates its index in MongoDB a moment later. This makes queries such as "rooms whose status is not NORMAL" work in `query_twins.py`.

25

library-edge-gateway

Node "Check result". On 204 it adds 1 to the **twinUpdates** counter and shows "twin updated (R02)" under the node. On any other answer it logs a warning.

The twin `library:R02` now holds the **latest state of the Computer Lab**.

Phase 6 • Gateway statistics (every 5 seconds)

26

`library-edge-gateway`

Nodes "every 5 s" → "Data reduction stats". Reads the counters (raw, valid, dropped, windows, summaries, alerts, twin updates) and calculates **forwarded = summaries + alerts** and **reduction = 1 – forwarded / raw**.

27

Store and show. Sends a PATCH to the twin `library:gateway` (same path as steps 19–23), so the numbers are stored in Ditto, and sends the numbers to the "Edge gateway" box on the dashboard.

Phase 7 • From the twin to the dashboard

28

`library-edge-gateway`

Nodes "every 3 s" → "GET twins". Every 3 seconds the gateway builds **GET** `http://nginx/api/2/things?ids=library:R01,library:R02,library:R03` with the same login header.

29

`library-ditto-nginx`

`library-ditto-gateway`

`library-ditto-things`

Read the twins. Login check as in step 20, then a permission check: may this user **read** the twins? Yes. Things answers with all three twins as one JSON list, attributes and features included.

30

`library-edge-gateway`

Node "Format for UI". Turns each twin into a simple card object: id, name, floor, capacity, status, reasons, occupancy, %, bar width, temperature, humidity, the four devices, seats and last-update time. If Ditto cannot be reached, it shows an error message instead.

31

`library-edge-gateway`

Node "Rooms" (dashboard template). Node-RED Dashboard sends this data over a **websocket** to every browser that has `/ui` open.

32

`your browser`

Draw the cards. You open `http://localhost:1880/ui`; Docker forwards port 1880 to the gateway container. One card per room:

- Colour and badge come from the status: **green NORMAL**, **orange WARNING**, **red CRITICAL**.
- Occupancy bar, temperature and humidity, device chips (a FAULT chip turns red), Seats, and the "Why" line.

The popup from step 15b and the alert list arrive on the same websocket.

Done. About **10–14 seconds** pass from a sensor reading to a colour change on screen: about 10 s to fill a window (one reading per second, minus dropped ones), plus up to 3 s for the next dashboard refresh.

What each part does, in one line

Module / container	Its one job
<code>library-sensors</code>	Pretends to be the room sensors and publishes readings

library-mqtt-broker	Post office: delivers MQTT messages by topic
library-edge-gateway	The brain at the edge: cleans, converts, groups, decides, alerts, updates twins, serves the dashboard
library-ditto-nginx	Security guard: checks the login before anything reaches Ditto
library-ditto-gateway	Receptionist: Ditto's REST API, turns HTTP into internal commands
library-ditto-things	Keeper of the twins: stores and updates each room's state
library-ditto-policies	Rule book: who may read or write which twin
library-ditto-search	Index: answers search questions over all twins
library-ditto-mongodb	Filing cabinet: the database under Ditto
library-twin-setup	Creates the twins once at start-up, then exits
Helper scripts (run on demand)	<code>query_twins.py</code> reads twins (GET and search); <code>compare_traffic.py</code> counts raw vs forwarded messages; <code>pytest</code> checks the whole chain